SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

ASSESSMENT TOOL III - ASSIGNMENT

COURSE CODE: ITA14 **COURSE NAME:** Ethical Hacking

COURSE OUTCOME COVERED

CO1: *Apply the principle of the foundational concepts of ethical hacking, including types of hackers, ethical principles, legal frameworks, and the role of responsible disclosure in cybersecurity.*

ASSIGNMENT QUESTION

3. During a black-box penetration test, you accidentally trigger a denial-of-service condition on a live production server, causing an outage. Walk through what you should have done to prevent this and what your responsibility is now.

Name: Avinash M

Reg No: 192521196

1. Understanding the Scenario

In this question I am working as a penetration tester on a black-box engagement. Black-box testing means that the client gives me almost no internal information about their systems. I do not get network diagrams, source code or login details. I only know what a real outside attacker would know, which is usually a company name, a domain or a range of IP addresses. My job is to find weaknesses from that outside position and report them so that they can be fixed before a criminal finds them.

During the test, one of my actions accidentally causes a denial-of-service (DoS) condition on a live production server. A denial-of-service condition simply means that a system can no longer answer genuine users, either because it has been overloaded with requests or because the service has crashed. A production server is the real server that the company uses for its daily business, so it is not a practice machine. When it goes down, real customers cannot log in, real orders may fail and the company may lose money and trust.

The question asks me two things. First, what I should have done before and during the test to stop this from happening. Second, what my responsibility is now that it has happened. In my answer I connect both parts to the topics of CO1, namely the types of hackers, ethical principles, legal frameworks and responsible disclosure.

Summary: *A black-box tester knows very little about the target, so extra care is needed. A DoS on a live server causes real harm, and the question is about both prevention and the tester's duty after the damage.*

2. Ethical Hacking Principles Involved

2.1 Types of hackers

Hackers are usually grouped by their permission and their intention. A white hat hacker has permission and works to improve security. A black hat hacker has no permission and acts for personal gain or to cause harm. A grey hat hacker sits in between, because they act without permission but usually without a harmful goal. I am a white hat in this scenario because the client hired me.

However, one important point is that being a white hat does not mean that nothing can go wrong. The tools I use can damage systems in the same way whether I am hired or not. What separates me from a black hat after an accident is how I behave next: whether I stop, admit it, and help the client recover.

2.2 Core ethical principles

- **Authorization.** I may only test what the client has agreed to in writing. Permission is the starting point of ethical hacking.
- **Scope.** Authorization has limits. Only the listed systems, methods and time periods are allowed.
- **Do no harm.** I should reduce the chance of damage to the lowest level possible, especially on live systems.
- **Honesty and transparency.** If I cause a problem, I must tell the client quickly and truthfully.
- **Confidentiality.** Everything I see during the test, including the incident, must stay private between me and the client.
- **Professional accountability.** I am responsible for my own actions and for the tools I run, even if the tool did the work automatically.

Summary: *There are white, black and grey hat hackers, and the real difference is permission, intent and conduct. The principles of authorization, scope, do no harm, honesty, confidentiality and accountability all apply directly to this outage.*

3. Why the Outage Probably Happened

Before talking about prevention, it helps to understand how a tester can cause a DoS by accident. In real engagements this is not rare. Some common reasons are:

- Scanning too fast. Sending thousands of packets per second can overwhelm an old or small server.
- Running a vulnerability scanner with intrusive or dangerous checks turned on. Some of these checks are designed to crash a service in order to prove it is vulnerable.
- Using an exploit that is known to crash the service if it fails, such as a memory corruption exploit.
- Brute-force or fuzzing attempts that open a very large number of sessions or fill up the database.
- Testing during busy business hours, when the server is already close to its limit.
- Not knowing that the server was fragile, outdated or a single point of failure for the whole company.

Because this is a black-box test, I could not know most of these facts in advance. That is exactly why the correct attitude is caution. When I know little about a system, I should assume it can break easily.

Summary: *Outages usually come from aggressive speed, intrusive checks, crash-prone exploits, heavy fuzzing, bad timing or unknown weak servers.*

4. What I Should Have Done to Prevent It

4.1 Before the engagement starts

Most of the prevention work happens before I send a single packet. A one-line email authorization is not enough. I should have asked for a proper document, usually called the Rules of Engagement (RoE), and checked the following points with the client:

- The exact scope: which IP addresses, domains and servers are included, and whether live production systems are included at all.
- Whether denial-of-service or stress testing is allowed. If it is not clearly allowed, I must treat it as forbidden.
- Testing windows, for example late at night or on weekends, so that any problem affects fewer people.
- A list of fragile or business-critical systems that need extra care or should be left alone.
- A 24x7 emergency contact (a name and phone number) and the authority to stop the test at any time.
- Confirmation that the client has recent backups and a way to restore services.
- The source IP addresses I will use, so that the client's security team can recognise my traffic and not confuse it with a real attack.
- A contract that states clearly who is responsible if something goes wrong, and whether I have professional liability insurance.

If the client would not agree to any of this, a sensible tester would ask for a copy of the system in a test environment, or would refuse to test the live server.

4.2 During the test

Even with a good plan, the way I run the test matters a lot. I should have begun slowly and increased intensity step by step instead of using full speed from the start. For example, a port scan can be slowed down with options such as a lower timing template or a maximum packet rate, so that the scan does not flood the server. Vulnerability scanners should be set to “safe checks only”, and any exploit that is known to crash services should be avoided on production or saved for a separate, approved test.

I should also have watched the target while testing. A simple habit is to check every few minutes whether the website or service still responds normally and how fast. If the response time starts to climb or the service becomes unstable, I should pause immediately instead of continuing. Keeping a time-stamped log of every command I run is also important, because it lets me tell the client exactly what happened if there is a problem.

4.3 Prevention checklist

Stage	Action	Reason
Planning	Get signed Rules of Engagement with scope and DoS rules	Removes confusion about what is allowed
Planning	Agree on testing hours and emergency contacts	Reduces impact and allows quick reaction
Planning	Confirm backups and ask about fragile systems	Makes recovery possible
Testing	Start with low speed and safe checks only	Avoids overloading the server
Testing	Avoid crash-prone exploits on production	A failed exploit can crash the service
Testing	Monitor response time and stop at first sign of trouble	Catches a problem before it becomes an outage
Testing	Log every command with time	Gives accurate evidence for the report

Summary: *Prevention means getting proper written rules first, then testing slowly, watching the target and stopping early. A one-line email was not a safe basis for testing a live server.*

5. What My Responsibility Is Now

The outage has already happened, so what matters now is how I react. A client can forgive an honest mistake, but not a hidden one. I would follow these steps in order.

1. **Stop testing at once.** I must pause every scan and tool immediately so that I do not make the damage worse.
2. **Inform the client right away.** I should use the emergency contact from the Rules of Engagement and not wait until the final report. I should be

honest that my activity is the likely cause and give the facts: the time, the target, the tool or command used and my source IP address.

3. **Do not hide anything.** I must never delete logs, change timestamps or pretend that the outage was a coincidence. I should keep my own logs and screenshots safe, because they help the client find the cause.
4. **Support recovery within my authorization.** I can give the client's administrators the details they need. I should not restart or repair the server myself unless I am asked to do so in writing, because that would go beyond my scope and might destroy useful evidence.
5. **Wait for permission before continuing.** I should only resume testing after the client has agreed in writing, and I should propose a safer plan such as slower scans or a test copy of the server.
6. **Write an incident report.** The report should explain what happened, when, why, how long the service was affected and what will be changed to prevent it again.
7. **Accept accountability.** I should not blame the client for having a weak server. I should also inform my team lead or employer, and accept any responsibility defined in the contract, which may include compensation.

5.1 Example of the first message to the client

"This is Avinash M from the penetration testing team. At about 2:15 AM, while running a scan against the main web server, the server stopped responding. I believe my scan caused it. I have stopped all testing. The tool used, the command and my source IP are listed below. Please let me know how I can help your team and I will not continue until you approve."

5.2 What the incident report should contain

Section	What it includes
Summary	A short description of the incident and its business impact
Timeline	Exact times of the test action, the outage, the notification and the recovery
Technical details	Tool, command or payload, target and source IP
Root cause	Why the action led to the outage, for example scan speed or a crash-prone check
Impact	Services affected, duration and any possible data or customer effect
Recommendations	Fixes for the server, such as rate limiting and better monitoring, and changes to future testing

Summary: *My responsibility is to stop, report honestly and quickly, preserve evidence, help only within my authorization, wait for approval, document everything and accept accountability.*

6. Legal Framework

Written authorization is what makes penetration testing legal, but it only covers the activities that were actually agreed. Different countries have laws about damaging computer systems. In India, the Information Technology Act, 2000 deals with this in Section 43, which covers damage to a computer system and includes denying authorised users access to it, and Section 66, which makes such acts a criminal offence when they are done dishonestly or fraudulently. In the United States, the Computer Fraud and Abuse Act (18 U.S.C. 1030) covers damage to protected computers, and in the United Kingdom the Computer Misuse Act 1990 covers unauthorised acts that impair the operation of a computer.

For my situation, the legal position depends heavily on my behaviour. If I had written permission, acted in good faith and reported the outage honestly, the risk is much lower and the matter is mostly about the contract. If I went outside the

scope, hid the incident or tried to cover it up, my position becomes far worse. I am a student and not a lawyer, so the exact liability would depend on the contract and the laws of the place where the client is located.

Summary: *Authorization protects me only for agreed actions. Acting honestly and within scope reduces both legal and contractual risk.*

7. Responsible Disclosure and Confidentiality

The outage is not only a mistake, it is also a finding. It shows that the server can be knocked offline by a certain kind of traffic or requests, and an attacker could do the same thing on purpose. So this weakness should be included in my report to the client as a security issue.

Responsible disclosure means that I report the weakness privately to the owner, give them reasonable time to fix it, and do not share the details publicly. I should not post about the incident on social media, discuss it with outsiders or use it to show off, and I should respect any non-disclosure agreement. My recommendations could include rate limiting, a web application firewall, load balancing, extra capacity, redundancy and proper monitoring with alerts. Once the client has made changes, I can offer to re-test the system with their approval.

Summary: *The outage reveals a real weakness. I should report it privately with clear fixes and keep the details confidential.*

8. Ethical and Unethical Responses Compared

Situation	Ethical response	Unethical response
Server stops responding	Stop testing and call the client	Keep scanning or pretend nothing happened
My logs	Preserve them as evidence	Delete or edit them
Fixing the server	Ask for written permission first	Log in and change things without approval
Talking to the client	Admit the likely cause honestly	Blame the client's old server
The weakness found	Report it privately with advice	Publish it or boast about it

9. Real-Life Relevance and Lessons

Situations like this appear in real security work. Automated scanners are known to crash old network printers, medical equipment, industrial controllers and ageing web servers that were never built to handle unusual traffic. For banks, hospitals and online shops, even a short outage can have serious consequences. This is why professional standards such as the NIST guide on security testing, the Penetration Testing Execution Standard and the EC-Council Code of Ethics all stress planning, permission and careful testing.

The main lessons I take from this scenario are:

- Never depend on a vague authorization. Always get clear written rules of engagement.
- Be more careful when I know less about the target.
- Understand what a tool does before running it on a live system.
- If something goes wrong, speak up immediately and honestly.

10. Conclusion

Accidentally causing an outage during a black-box test is a serious event, but it does not automatically make someone a bad ethical hacker. What decides that is the preparation before the test and the behaviour after the accident. I should have asked for proper rules of engagement, avoided intrusive actions on a live system, tested slowly and monitored the server. Now that the damage is done, I must stop, inform the client truthfully, keep evidence, support recovery within my permission, report the weakness responsibly and accept accountability. Following these steps keeps the work within the principles of ethical hacking and protects the trust that the profession depends on.

References

1. NIST Special Publication 800-115, Technical Guide to Information Security Testing and Assessment.

2. Penetration Testing Execution Standard (PTES), pre-engagement interactions.

3. EC-Council Code of Ethics.

4. The Information Technology Act, 2000 (India), Sections 43 and 66.

5. Computer Fraud and Abuse Act, 18 U.S.C. 1030, and the Computer Misuse Act 1990 (UK).

Rubrics

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all data-handling requirements and operations	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate data structures and algorithms to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct code, output and complexity analysis with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand